

Appliance Energy Prediction

Multivariate Time-Series Deep Learning - Take-Home Assessment

Muaadh Nazly

2026-07-06

Contents

1	Introduction	3
2	Data Insights	4
2.1	Dataset Overview	4
2.2	Target Variable	4
2.3	Seasonality (STL Decomposition)	4
2.4	Temporal Patterns	8
2.5	Correlation Analysis	8
2.6	Feature Distributions	8
2.7	Outlier Detection	8
2.8	ACF / PACF Analysis	10
2.9	EDA Summary Table	10
3	Preprocessing	12
3.1	Missing Values and Duplicates	12
3.2	Noise Column Removal	12
3.3	Outlier Treatment	12
3.4	Scaling	12
3.5	Data Splitting	12
4	Feature Engineering	13
4.1	Time-Based Features	13
4.2	Rolling Averages and Moving Windows	13
4.3	Lagged Features	13
4.4	Interaction Features	13
4.5	Domain-Specific Features	13
4.6	Handling Engineering-Induced Missing Values	13
4.7	Feature Selection: A Four-Stage Process	14
5	Model Design	16
5.1	Baselines	16
5.2	Deep Learning Architectures	16
5.3	The Hybrid Model Found During Optimization	17

6	Results	18
6.1	Baseline Comparison	18
6.2	Deep Learning Comparison	18
6.3	Residual Diagnostics	19
6.4	Attention Weight Interpretation	20
6.5	Final Ranking Across All Models and Optimization Variants	20
7	Model Optimization	23
7.1	Standard Optimization Techniques	23
7.2	Investigating Why Deep Learning Underperforms	23
8	Challenges and Solutions	26
9	Conclusion	28
10	References	30

1 Introduction

Repository: <https://github.com/Muaadh-Nazly/energy-prediction>

This report covers the full pipeline built to predict household appliance energy consumption (**Appliances**, measured in Wh) using the UCI Appliance Energy Prediction dataset. It contains environmental, time-based, and energy-related sensor readings taken every 10 minutes from a low-energy house in Belgium, from January to May 2016.

The goal was to preprocess and analyze the dataset, engineer useful features, and design architectural models that can forecast **Appliances** consumption, using a proper, leakage-free evaluation setup.

The work is spread across five notebooks and a shared `src/` library:

1. **EDA** (`01_EDA.ipynb`): data quality checks, target distribution, seasonality, correlation, and scaling analysis
2. **Feature Engineering** (`02_Feature_Engineering.ipynb`): cyclical, rolling, lag, and interaction features, plus a four-stage feature selection process
3. **Baseline Models** (`03_Baseline_Models.ipynb`): Linear Regression and Random Forest
4. **Deep Learning Models** (`04_Deep_Learning.ipynb`): LSTM, GRU, CNN-LSTM, TCN, and CNN-LSTM with Attention
5. **Optimization and Evaluation** (`05_Optimization_Evaluation.ipynb`): hyperparameter tuning, ablation studies, comparisons against classical and tree-based models, and a hybrid model built from two of the above

Before going further, it would be better to be upfront about the headline result because it isn't the one someone would expect from a brief that's framed entirely around deep learning: Linear Regression beats every standalone deep learning architecture that was trained, by a wide margin, and closes almost all of the remaining gap to a simple univariate ARIMA model with no feature engineering at all. Only a hybrid built specifically to learn what Linear Regression gets wrong, a CNN-LSTM trained on its residuals, edges past it, and only by a small margin. Section 8 walks through the tests ran to check whether that gap is something which could still fix, or whether it's a genuine limit set by the data itself. Section 10 explains why that finding, and the way it was investigated, is actually the most useful result this assessment produced.

2 Data Insights

2.1 Dataset Overview

Property	Value
Rows	19,735
Columns (raw)	29
Date range	2016-01-11 17:00 to 2016-05-27 18:00
Duration	137 days
Interval	10 minutes
Missing values	None
Duplicate rows	0

The raw dataset doesn't quite match the assessment's description. It has 9 indoor temperature and humidity sensor pairs (T1 to T9, RH_1 to RH_9) instead of 6, plus a **Tdewpoint** column, and it's missing the **NSM**, **WeekStatus**, and **Day_of_week** columns the brief mentions. Those three were engineered from the timestamp instead (see Feature Engineering below). The dataset also includes two random noise columns, **rv1** and **rv2**, which the original dataset authors added on purpose as a control to test whether feature selection would correctly discard them. It was confirmed that they're identical to each other and barely correlated with the target (correlation of -0.011), so both were dropped right away.

2.2 Target Variable

Appliances is heavily right-skewed:

Statistic	Value
Mean	97.69 Wh
Std	102.52 Wh
Median	60.0 Wh
Min / Max	10 / 1080 Wh
Skewness	3.39
Kurtosis	13.67

This skew is the reason Huber loss was used instead of plain MSE for every deep learning model (more on this in Preprocessing). MSE would let the loss be dominated by rare high-consumption spikes rather than typical behavior.

2.3 Seasonality (STL Decomposition)

Running an STL decomposition with a period of 144 steps (24 hours) shows a strong daily seasonal cycle, with the seasonal component ranging from -285.1 to +733.9 Wh against a much smaller trend component (38.5 to 115.0 Wh). This is what justified using 144 steps as one of the candidate sequence windows tested later for the deep learning models.

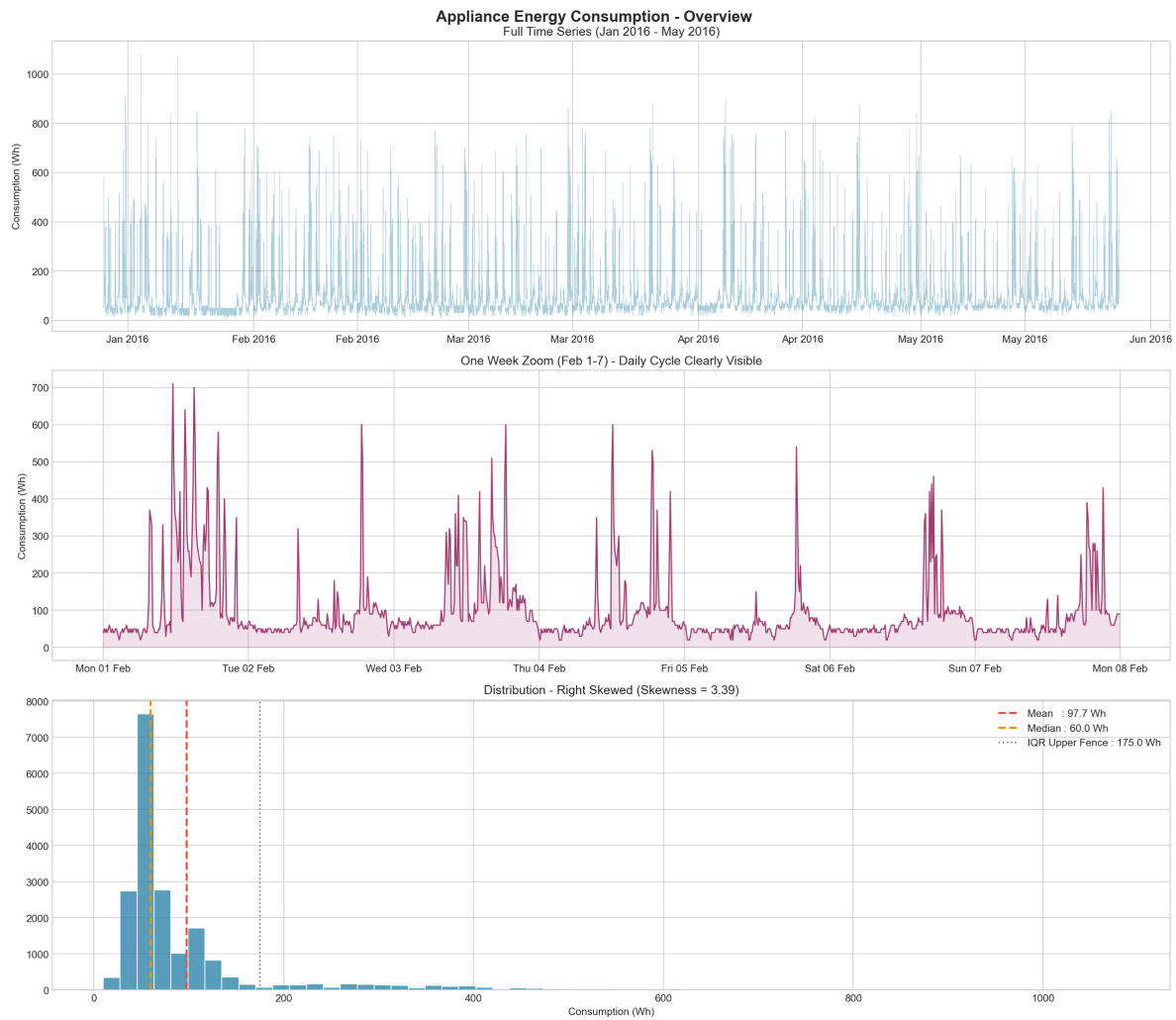


Figure 1: Target distribution and time series overview



Figure 2: STL decomposition: trend, seasonal, residual

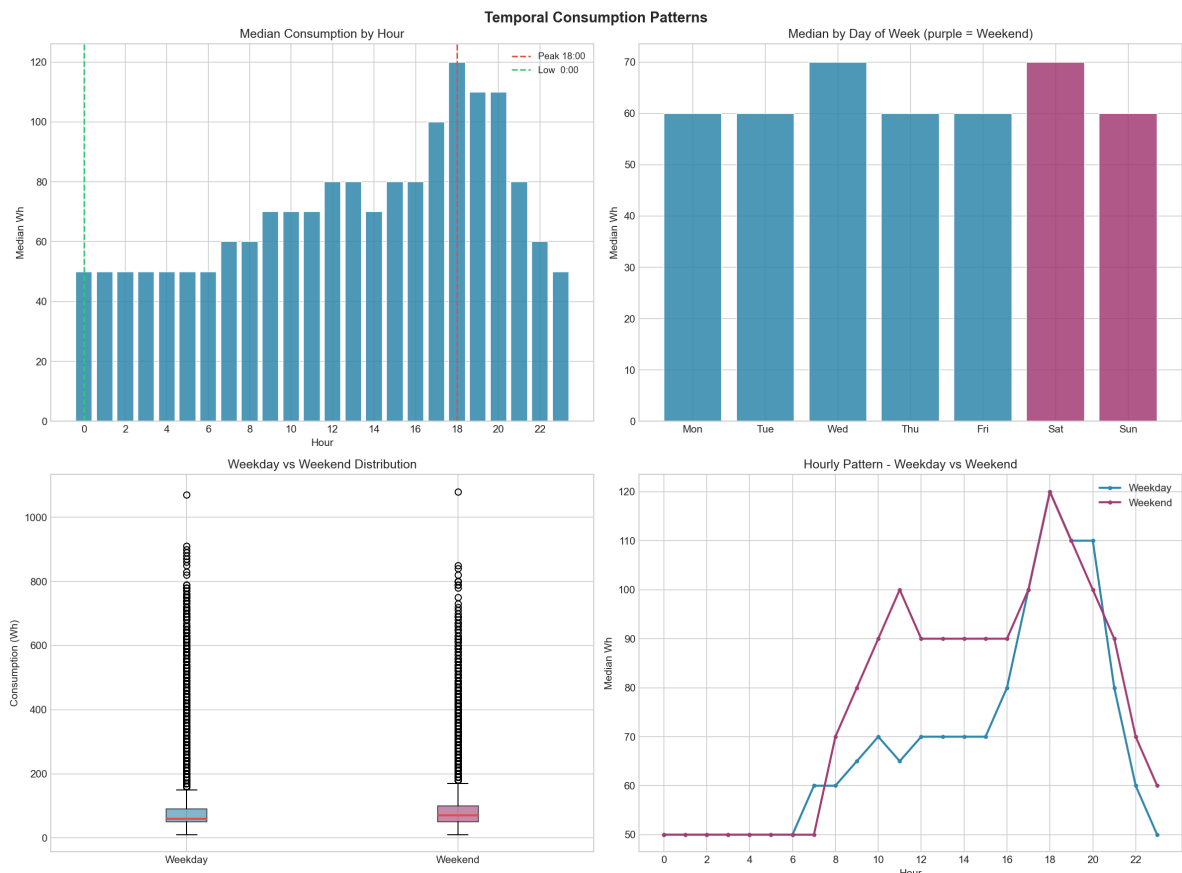


Figure 3: Hourly, weekday/weekend, and monthly consumption patterns

2.4 Temporal Patterns

A few things stood out here:

- Peak hour is 18:00 (120 Wh average), and the lowest point is 00:00 (50 Wh average)
- Weekday average is 96.6 Wh, weekend average is 100.6 Wh, so there's a modest but real difference between the two
- Weekends show a consumption bump between 11:00 and 12:00 that doesn't show up on weekdays at all. That pattern doesn't exist in any raw column on its own, so a `weekend_lunch_peak` feature was built specifically to capture it

2.5 Correlation Analysis

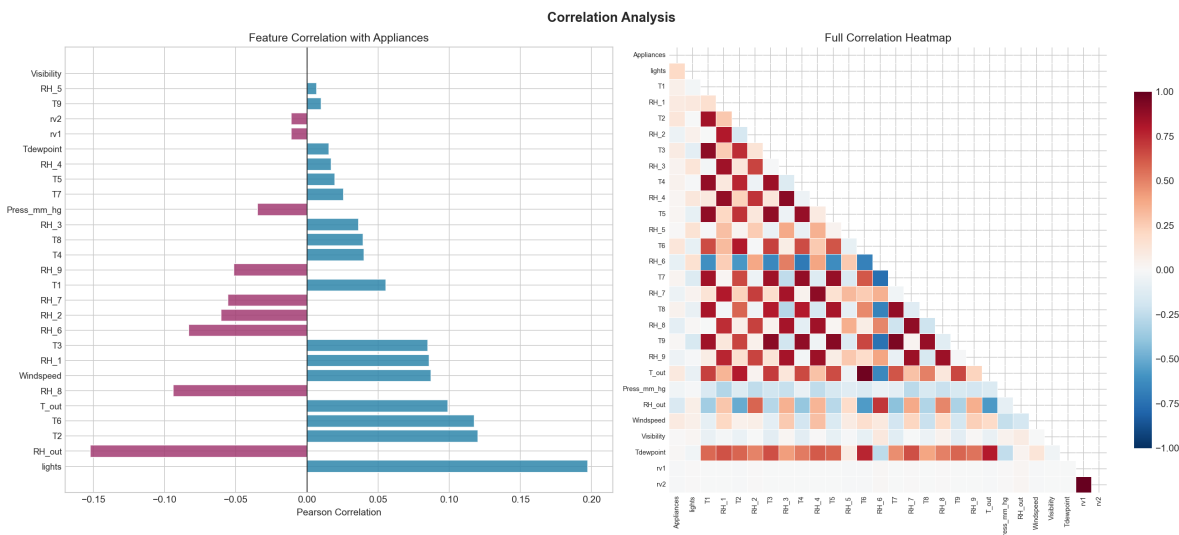


Figure 4: Feature correlation with the target

No single raw sensor correlates strongly with **Appliances** by itself. The strongest ones are **lights** (0.197), **RH_out** (-0.152), **T2** (0.120), and **T6** (0.118), and none of those are particularly strong. This turned out to be an early hint of something bigger, that would be discussed in Model Optimization: **Appliances** is much better predicted by its own recent history than by any environmental sensor.

2.6 Feature Distributions

Most of the indoor temperature and humidity sensors (T1 to T5, T7 to T9, RH_1 to RH_9) look roughly normal, which is why the pipeline went with StandardScaler for the input features. T6 is the exception. It looks flat and uniform, sitting between -5°C and 12°C rather than following a normal curve. Later correlation analysis (in Feature Engineering, Stage 2) confirms T6 is basically a duplicate of the outdoor temperature T_out (correlation of 0.975), which fits with it being an outdoor-adjacent sensor rather than a genuine indoor reading.

2.7 Outlier Detection

Using the standard IQR method (Q1=50, Q3=100, IQR=50), 2,138 rows (10.83% of the data) go above the upper fence of 175 Wh. It was decided to clip these rather than remove them. Removing rows from a time series leaves gaps in the index, and any rolling or lag feature computed afterward

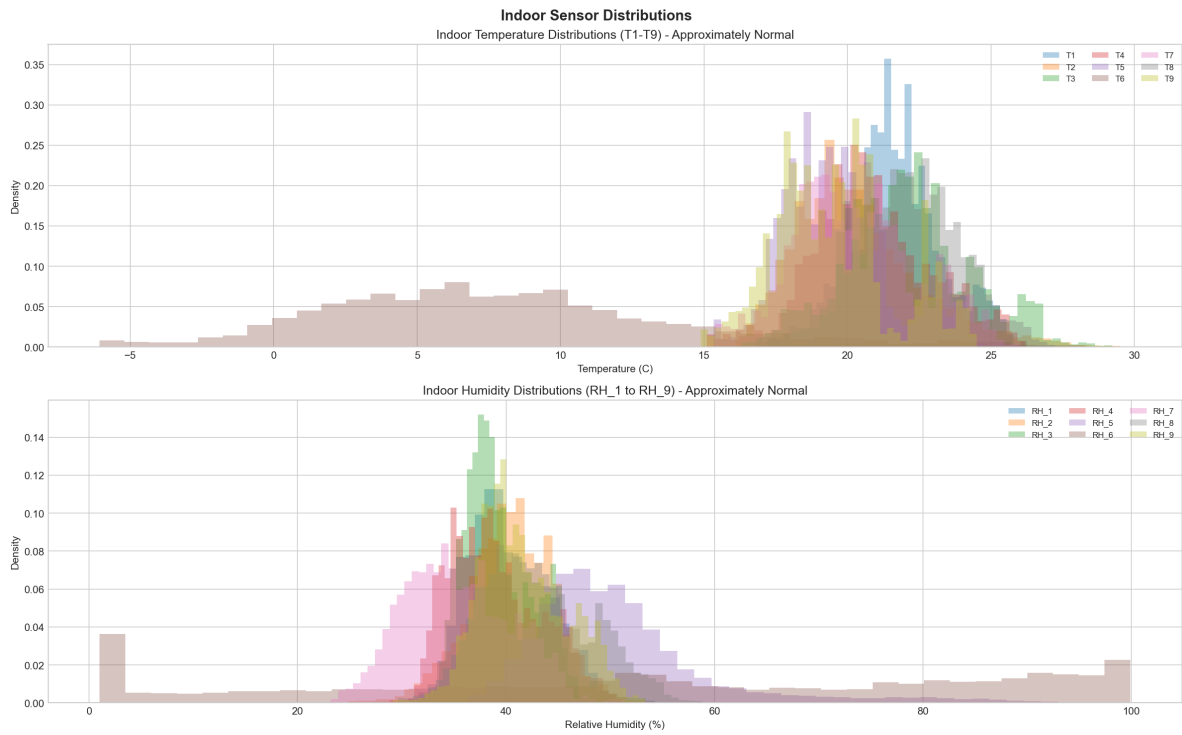


Figure 5: Indoor sensor distributions

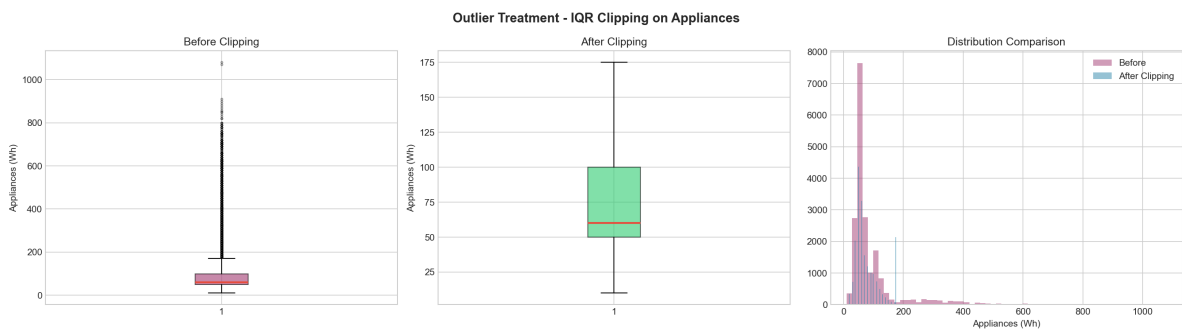


Figure 6: Outlier treatment via IQR clipping

would either produce NaNs or accidentally splice together timestamps that aren't actually adjacent. Clipping keeps every row in place while capping the extreme values.

2.8 ACF / PACF Analysis

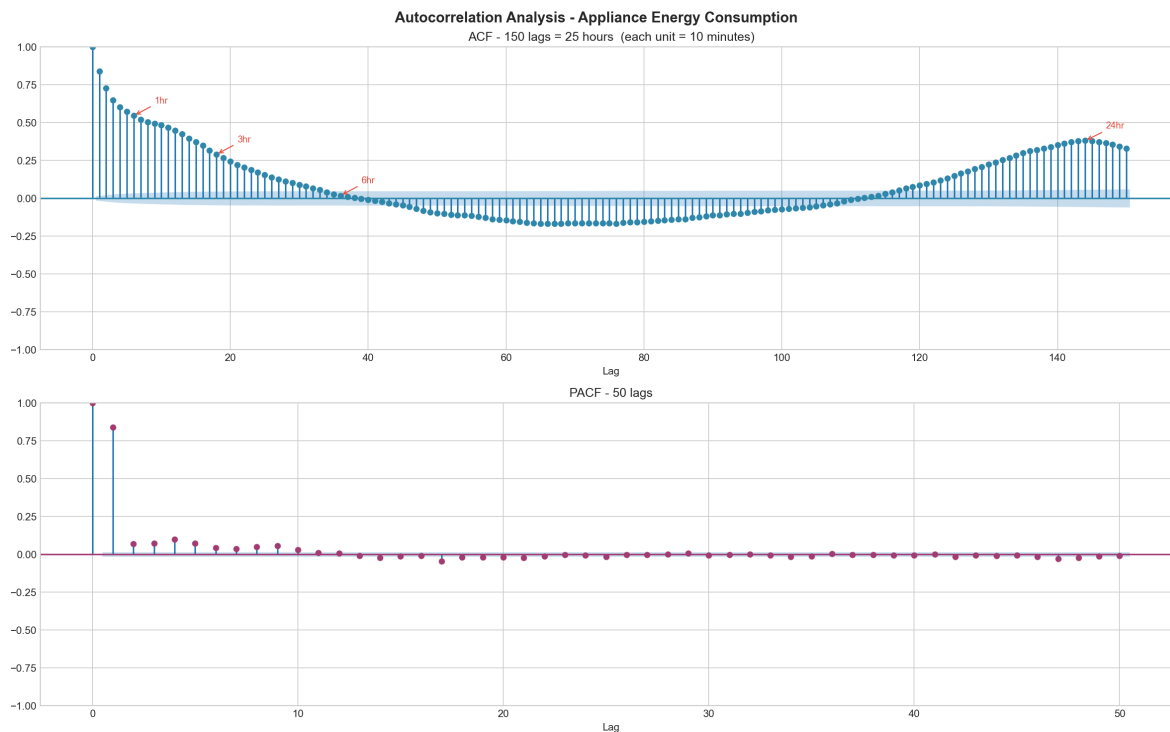


Figure 7: Autocorrelation and partial autocorrelation of the target

Lag	Duration	ACF
1	10 min	0.8405
3	30 min	0.6482
6	1 hr	0.5467
18	3 hr	0.2913
36	6 hr	0.0193
144	24 hr	0.3818

The PACF plot is significant almost only at lag 1 and 2, which is a textbook AR(1) signature. The ACF plot forms a U-shape, dropping toward lag 36 and then climbing back up at lag 144, confirming the 24-hour cycle. This analysis is what shaped the final lag set used in Feature Engineering (1, 3, 6, 18, 36, 144) and the three candidate sequence windows tested for the deep learning models (24, 72, 144 steps).

2.9 EDA Summary Table

Finding	Decision
No missing values, no duplicates	No imputation or deduplication needed
rv1/rv2 identical, near-zero target correlation	Dropped
Extra sensors (T7 to T9, RH_7 to RH_9, Tdewpoint) present	Kept, since they add signal
NSM/WeekStatus/Day_of_week absent	Engineered from the timestamp
Target skewness 3.39	Huber loss as the main loss, MSE tested for comparison
2,138 outlier rows (10.83%)	Clipped at the IQR upper fence, rows not removed
Sensors mostly normal, T6 uniform and outdoor-adjacent	StandardScaler for all input features
Target needs a bounded range	MinMaxScaler for Appliances only, fit after clipping
Strong daily seasonality (STL)	144-step window kept as one sequence-length option
PACF significant only at lag 1-2	Short lags carry most of the individual signal
ACF and PACF together	Lags 1, 3, 6, 18, 36, 144 selected
Weekend 11:00-12:00 peak absent on weekdays	Added a weekend_lunch_peak interaction feature

3 Preprocessing

All of the preprocessing logic lives in `src/data_preprocessing.py` and gets imported into each notebook rather than rewritten each time.

3.1 Missing Values and Duplicates

There weren't any. It was checked programmatically with `check_data_quality()` rather than just assuming it, so no imputation or deduplication step was needed.

3.2 Noise Column Removal

`rv1` and `rv2` are identical to each other and barely correlated with the target (-0.011), so `drop_noise_columns()` removes both. These are intentional random controls the original dataset authors added to test whether a feature selection pipeline would correctly throw them out. The four-stage selection process described later confirms independently that they carry no useful signal.

3.3 Outlier Treatment

`clip_outliers_iqr()` clips `Appliances` at the IQR upper fence (175 Wh, using the standard 1.5x multiplier) instead of removing rows. This choice was made, because deleting rows breaks the temporal continuity that every rolling and lag feature depends on. Clipping caps the 2,138 extreme rows (10.83% of the data) while keeping the series intact.

3.4 Scaling

Two different scalers were used for two different reasons:

Input features use `StandardScaler`. Most sensor readings are roughly normally distributed, as shown in the EDA, and `StandardScaler` centers each feature at 0 with unit variance independently of the others.

The target uses `MinMaxScaler`, applied after outlier clipping so it ends up bounded in a clean 0 to 1 range. That matches the natural output range of the models and avoids any scale mismatch when computing the loss.

Both scalers are fit only on the training set and then applied to the test set without being refit. Fitting on the full dataset, test set included, would leak information from the test set into training. It's a common mistake and was checked carefully.

3.5 Data Splitting

The standard split of 80/20 was used based on time, not a random split. The first 80% of the chronologically sorted data is training data, and the last 20% is test data, with no shuffling involved. A random split would let future rows leak into training, both directly and through any lag or rolling feature that looks at neighboring rows, which would quietly inflate every metric reported. A further 15% from the end of the training set was carved out as a validation split, still in temporal order, used for early stopping and comparing hyperparameters. The test set stays fully untouched until the final evaluation.

4 Feature Engineering

This is implemented in `src/feature_engineering.py` and applied in `notebooks/02_Feature_Engineering.ipynb`

4.1 Time-Based Features

The following features were engineered from the timestamp, since the raw dataset doesn't have them. They are `hour` (0-23), `day_of_week` (0 for Monday through 6 for Sunday), `month`, `week_status` (Weekday or Weekend), and `NSM` (seconds since midnight)

Cyclical encoding was added for `hour` and `day_of_week`, turning each into a sin/cos pair. A plain integer encoding makes hour 23 and hour 0 look maximally far apart, when they're actually adjacent. Placing them on a circle instead fixes that and matches the daily cycle that STL and ACF both confirmed.

4.2 Rolling Averages and Moving Windows

1-hour, 3-hour, and 6-hour rolling mean and standard deviation of `Appliances` (6, 18, and 36 steps at 10-minute resolution) were computed. Every rolling window is shifted by one step before it's applied, so a feature at row `t` never includes the value being predicted at row `t` itself. Without that shift, a rolling mean over rows `t-5` through `t` would quietly leak the answer into the input.

4.3 Lagged Features

Lags of `Appliances` at 1, 3, 6, 18, 36, and 144 steps were added and chosen directly from the ACF/PACF analysis. Lags 1 to 3 cover the AR(1) structure PACF showed, and lag 144 covers the daily cycle confirmed by ACF.

4.4 Interaction Features

- `heat_index_proxy`: `T_out` multiplied by `RH_out`
- `indoor_outdoor_delta`: the average of the indoor temperature sensors minus `T_out`
- `weekend_lunch_peak`: set to 1 if `day_of_week` is Saturday or Sunday and `hour` is 11 or 12. This captures the weekend-only midday peak from the EDA that neither `week_status` nor `hour` alone can represent.

4.5 Domain-Specific Features

A holiday or special event calendar wasn't added, since this is a single house in Belgium over 137 days, and the dataset doesn't include a holiday indicator, so it would've had to source one externally. It was decided against fabricating a calendar without a verified source, since getting it wrong would introduce incorrect labels rather than useful signal. These are listed as future work in the Conclusion rather than skip it silently.

4.6 Handling Engineering-Induced Missing Values

Lag-144 needs 144 prior rows to exist, so the first 144 rows of the dataset can't have a complete feature set. These rows (about 0.73% of the data) were dropped instead of imputing them, since imputing a lag feature would mean inventing history that never happened.

4.7 Feature Selection: A Four-Stage Process

Instead of relying on one method, the features were run through four progressively stricter stages, each catching a different kind of redundancy or noise:

Stage	Method	Features In -> Out
1	Zero-signal removal (rv1 , rv2)	50 -> 48
2	Pairwise correlation redundancy (threshold 0.95)	48 -> 46
3	Random Forest impurity importance (ranking only)	46 -> 46
4	Permutation importance on held-out validation data	46 -> 36

Stage 2 dropped exactly two redundant pairs: **T_out/T6** (correlation 0.975, confirming **T6**'s outdoor-adjacent nature from the EDA) and **NSM/hour** (correlation 0.999, two encodings of essentially the same thing).

Stage 3, Random Forest importance, ranked **Appliances_lag1** far above everything else, at roughly 0.72 relative importance, which lines up with the AR(1) signature from PACF. The rolling statistics formed a clear second tier below it. **Press_mm_hg** ranked surprisingly high, in the top 8, which was flagged as suspicious rather than trusting outright.

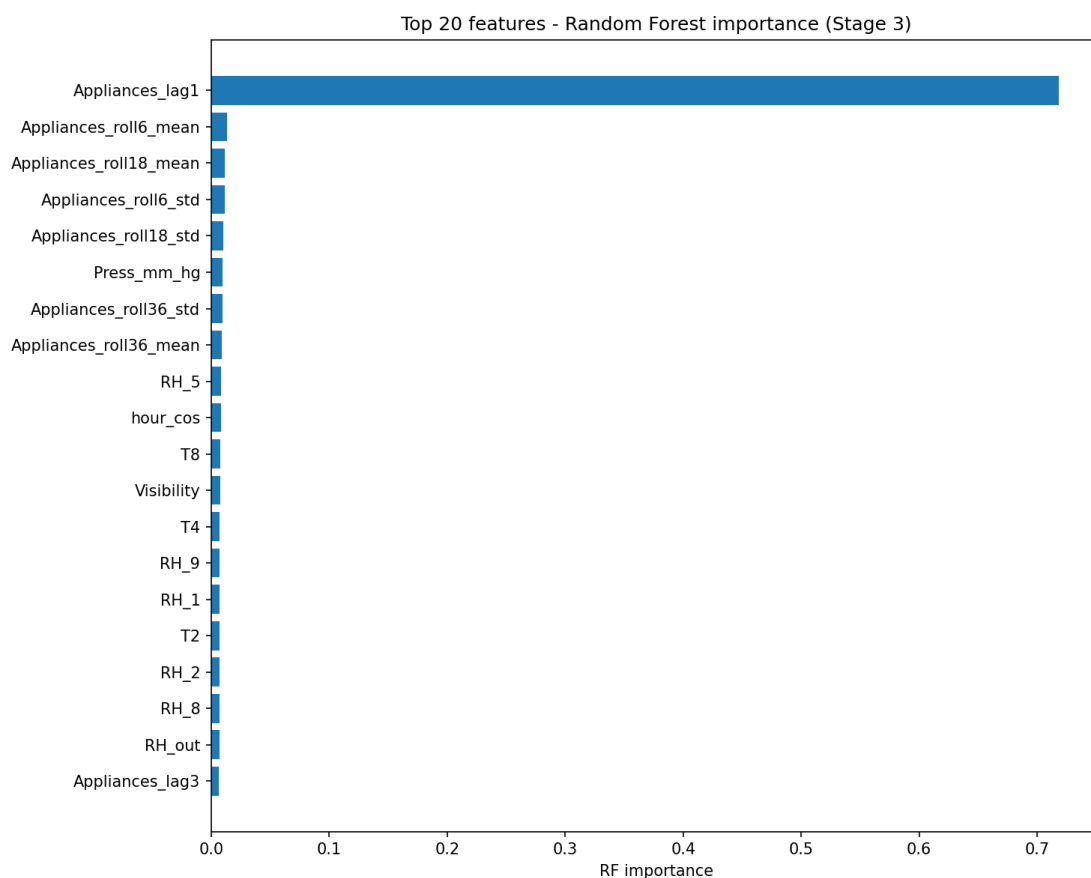


Figure 8: Random Forest feature importance

Stage 4, permutation importance, exists specifically because impurity-based importance is known to favor high-cardinality, continuous features. It measures importance on a validation split carved from the training data, not the test set, an important correction covered below. It confirmed **Appliances_lag1** (about 0.79) and **Appliances_rol16_mean** (about 0.025) as the two real drivers, with a steep drop-off after that. It also settled the **Press_mm_hg** question: once measured by actual held-out performance rather than tree impurity, it fell out of the top 20 entirely, confirming its Stage 3 ranking was an artifact rather than real signal.

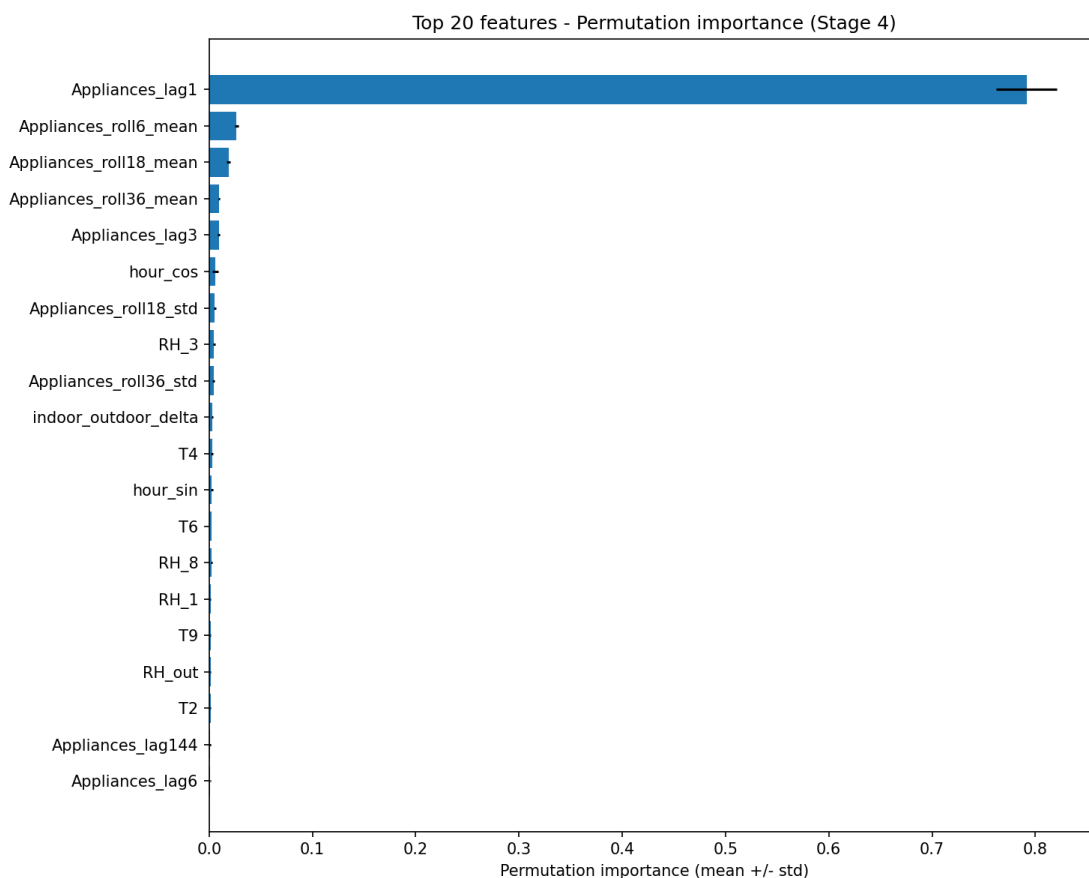


Figure 9: Permutation importance

Stage 4 measures importance on a validation split carved from training data specifically to keep the test set fully held out until final model evaluation (see Challenges and Solutions for why this mattered enough to call out separately).

The final set of 36 features is dominated by **Appliances** lag and rolling features (11 of the 36), humidity and temperature sensors (**RH_1** through **RH_9**, **RH_out**, and six of the **T1-T9** room sensors, 15 features total), the two interaction terms, cyclical and temporal features, and a handful of remaining raw sensors (**Tdewpoint**, **Press_mm_hg**, **Visibility**, **lights**). The full list is saved in `data/processed/selected_features.txt`.

5 Model Design

5.1 Baselines

Linear Regression has no hyperparameters to tune, so it doubles as a sanity check that the whole pipeline works end to end before moving on to anything more complex. Random Forest (`n_estimators=200`, `random_state=42`) is a non-linear baseline, using the same configuration already used for feature importance ranking so the numbers stay comparable.

5.2 Deep Learning Architectures

All five deep learning models are built in PyTorch (`src/model.py`) and share the same building blocks through a single `get_model()` function.

Each takes a sequence of the 36 selected features over some window of past time steps, with the window length chosen empirically (see Model Optimization).

For regularization, a custom function `SpatialDropout1D` module was built (using `nn.Dropout2d` over the feature-channel dimension) and applied it to the input sequence. Standard dropout zeroes individual values at each time step independently, which lets a recurrent layer patch over a dropped value using neighboring steps and weakens the regularization. Spatial dropout instead drops whole feature channels consistently across the entire sequence.

For the loss function, Huber loss was used by default, since the target’s skewness of 3.39 means MSE would be dominated by rare high-consumption spikes. Huber behaves like MSE for small errors and like MAE for large ones. An explicit MSE comparison was also run which is covered in Model Optimization.

For the optimizer, Adam with cosine annealing learning rate scheduling was used, which decays smoothly rather than dropping abruptly, lowering the chance of the optimizer getting stuck right after a sudden learning rate cut.

For training controls, early stopping was used based on validation loss, restoring the best-performing weights afterward, and fixed random seeds across `random`, `numpy`, and `torch` (including the training data loader’s shuffle order) so runs are reproducible.

Architecture	Structure	Hidden size	Notes
LSTM	Spatial dropout, then a single LSTM layer, then a linear layer on the last step	64	The baseline recurrent model
GRU	Same as LSTM but with a GRU cell	64	Fewer parameters than LSTM at similar capacity
CNN-LSTM	A Conv1D layer (32 channels, kernel size 3), then ReLU, spatial dropout, then an LSTM, then a linear layer	64	The convolution picks up local patterns before the LSTM handles longer-range dependencies

Architecture	Structure	Hidden size	Notes
TCN	Three stacked dilated causal blocks (dilations 1, 2, 4), each with two convolutions, a “chomp” step to trim padding, ReLU, dropout, and a residual connection	32 channels per layer	Gets a wide receptive field without needing a very deep stack
CNN-LSTM with Attention	Conv1D, spatial dropout, LSTM, then additive attention over every LSTM output step, producing a context vector fed into a linear layer	64	The main interpretability-focused model. Its attention weights are visualized in the Results section

The attention mechanism works by giving each time step a learned score, normalizing those scores with softmax so they sum to 1, and using them to build a weighted context vector. This lets both the model and me see which positions in the window it actually relies on, rather than only using the LSTM’s final hidden state the way CNN-LSTM does.

5.3 The Hybrid Model Found During Optimization

There’s a sixth configuration that wasn’t part of the original five architectures: Linear Regression, followed by a CNN-LSTM trained on Linear Regression’s residuals, with both predictions combined at inference time. This turns out to be the best performer overall, edging out Linear Regression alone. Its design and results are fully explained in Model Optimization and Results, since it came out of the optimization phase rather than the initial design.

6 Results

6.1 Baseline Comparison

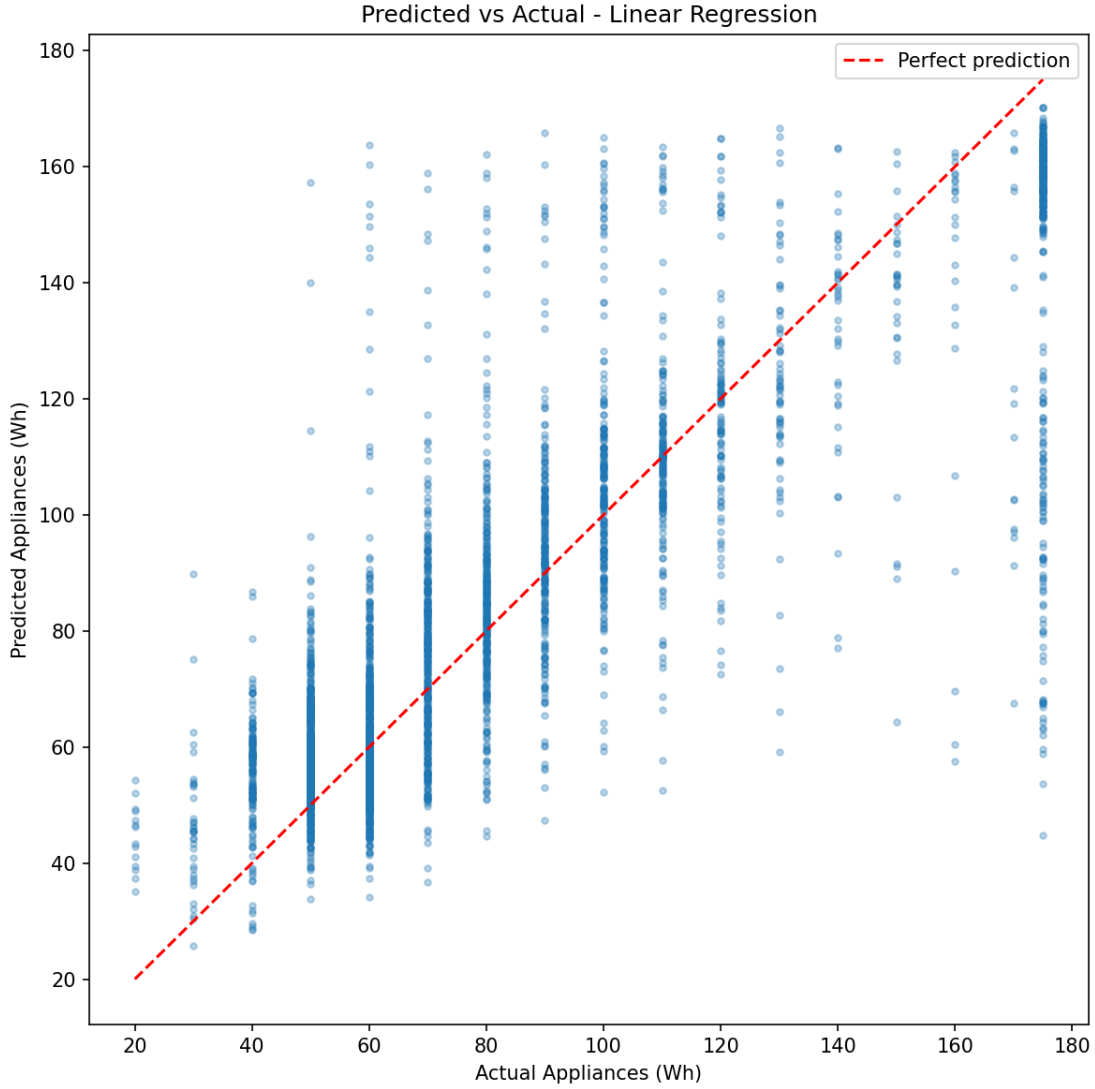


Figure 10: Baseline predicted vs. actual

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R2
Linear Regression	13.60	21.25	17.77	0.698
Random Forest	20.74	27.71	27.28	0.486

Linear Regression wins outright here. Random Forest’s default tree depth appears more sensitive to the feature set’s raw sensor columns than Linear Regression’s coefficients are, which lean almost entirely on the one dominant predictor, `Appliances_lag1`.

6.2 Deep Learning Comparison

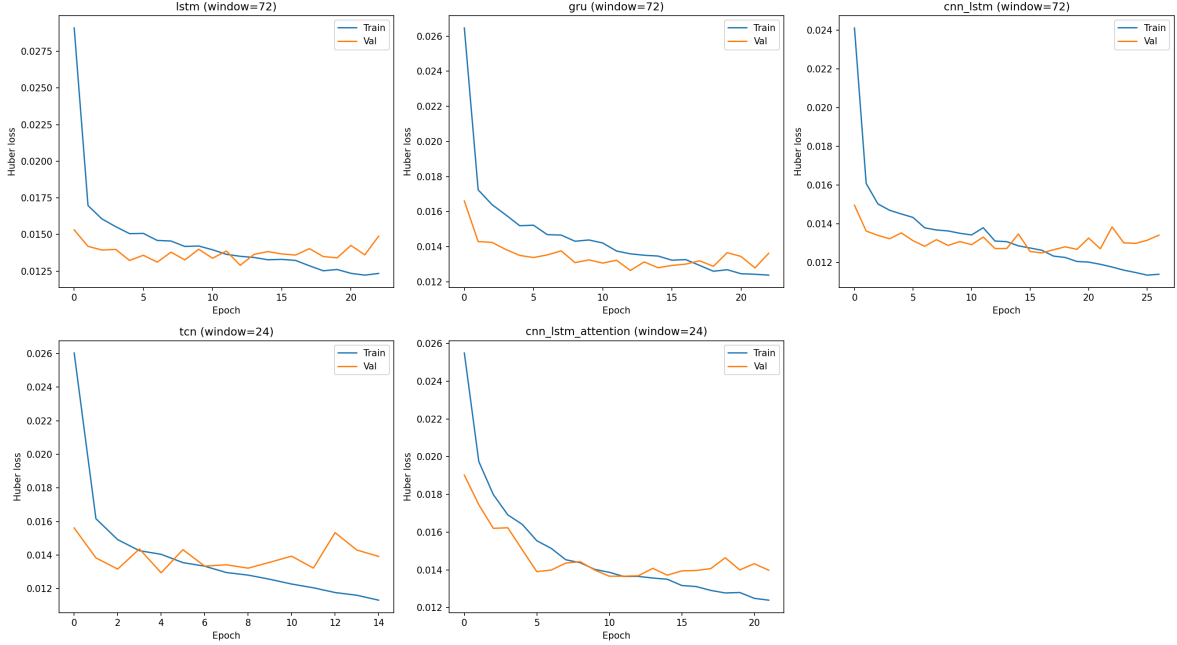


Figure 11: Learning curves across all 5 architectures

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R2	Window
CNN-LSTM	19.86	26.87	27.15	0.513	72
LSTM	20.43	28.11	27.97	0.467	72
TCN	20.75	28.18	28.41	0.463	24
GRU	21.63	28.88	29.92	0.437	72
CNN-LSTM with Attention	21.95	29.63	29.99	0.406	24

Three architectures (CNN-LSTM, LSTM, GRU) picked a 72-step (12-hour) window as best during the sweep, while TCN and CNN-LSTM with Attention picked 24 steps. The learning curves show healthy convergence for every architecture, where training loss decreases steadily, validation loss flattens out, and early stopping triggers before anything diverges. There's no overfitting problem here. The gap between train and validation loss stays small the whole time, which tells these models are underfitting the signal that Linear Regression already picks up directly, rather than overfitting or training badly.

CNN-LSTM is the strongest deep learning model of the five. Every architecture falls well short of both baselines.

6.3 Residual Diagnostics

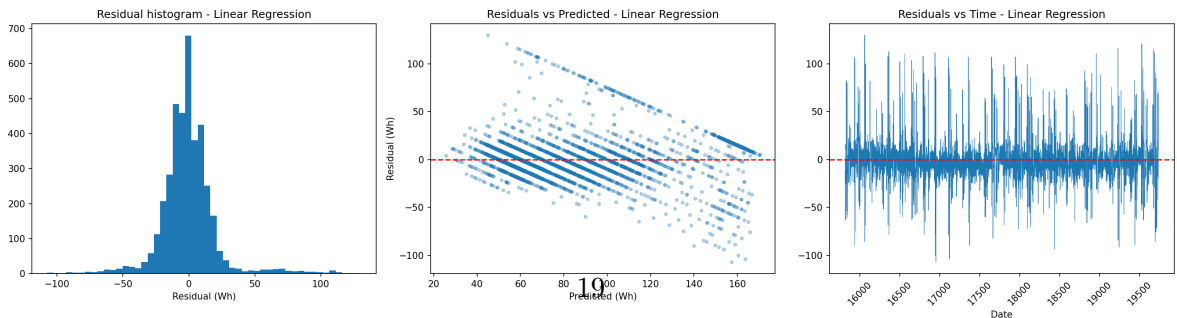


Figure 12: Residual diagnostics for Linear Regression

residual spread is visibly wider than Linear Regression's, matching its much higher RMSE, and its histogram sits noticeably left of zero rather than centered on it, a clearer negative bias on top of the wider spread. The overall shape of the error is still the same for both models, CNN-LSTM isn't failing differently, it's failing more, and more one-sidedly.

6.4 Attention Weight Interpretation

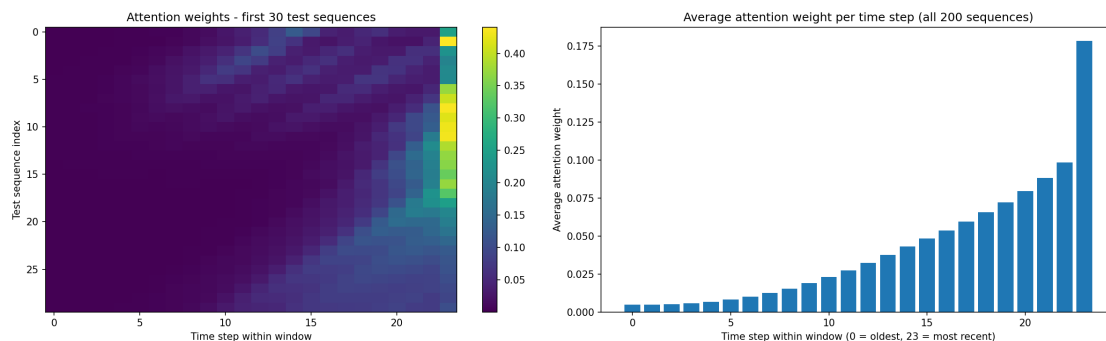


Figure 14: Attention weights across the 24-step window

Attention concentrates heavily on the single most recent time step. The average weight sits at 0.179 for the step right before the prediction, compared to well under 0.01 for the oldest steps in the window, and the most recent step is the single most-attended position in 156 of 200 sampled test sequences. This lines up with the AR(1)-like structure behind `Appliances_lag1`'s dominance throughout this project, and it matches the finding that the shortest candidate window (24 steps) beat the longer ones. The model has effectively learned on its own that recent history matters far more than distant history, which is exactly what the ACF/PACF analysis predicted back in the Data Insights section.

6.5 Final Ranking Across All Models and Optimization Variants

Model	MAE	RMSE	R2
Stacked (Linear Regression + CNN-LSTM on residuals)	13.52	21.08	0.700
Linear Regression	13.60	21.25	0.698
ARIMA(2,1,2)	13.37	21.64	0.686
Linear Regression (log target)	13.70	22.69	0.655
XGBoost (36 features)	16.45	22.78	0.652
CNN-LSTM (log target)	16.18	25.30	0.568
CNN-LSTM (MSE)	16.43	25.41	0.564
CNN-LSTM (extended patience)	17.84	25.79	0.551

Model	MAE	RMSE	R2
CNN-LSTM (batch_size=128)	17.69	26.09	0.541
CNN-LSTM (engineered v2 features)	18.60	26.25	0.535
CNN-LSTM (dropout=0.1)	17.84	26.35	0.531
CNN-LSTM (default)	19.86	26.87	0.513
CNN-LSTM (+ lagged exogenous sensors)	21.06	27.49	0.490
CNN-LSTM (tuned hyperparameters)	20.70	27.61	0.486
Random Forest	20.74	27.71	0.486
LSTM	20.43	28.11	0.467
TCN	20.75	28.18	0.463
GRU	21.63	28.88	0.437
CNN-LSTM with Attention	21.95	29.63	0.406

The full table is saved in `reports/final_results_table.csv`.

Plotted rather than just tabulated, the pattern is visible at a glance: the Stacked model and Linear Regression sit essentially tied at the top (RMSE 21.1-21.2), ARIMA and Linear Regression on a log target close behind (21.6-22.7), then a distinct gap down to XGBoost (22.8) and the full cluster of CNN-LSTM variants and other deep learning architectures (25.3-29.6). Of everything tried on CNN-LSTM standalone, only the log-transform and extended-patience experiments meaningfully closed part of that gap; nothing standalone reaches the top band. Only combining Linear Regression with a CNN-LSTM trained on its residuals gets there.

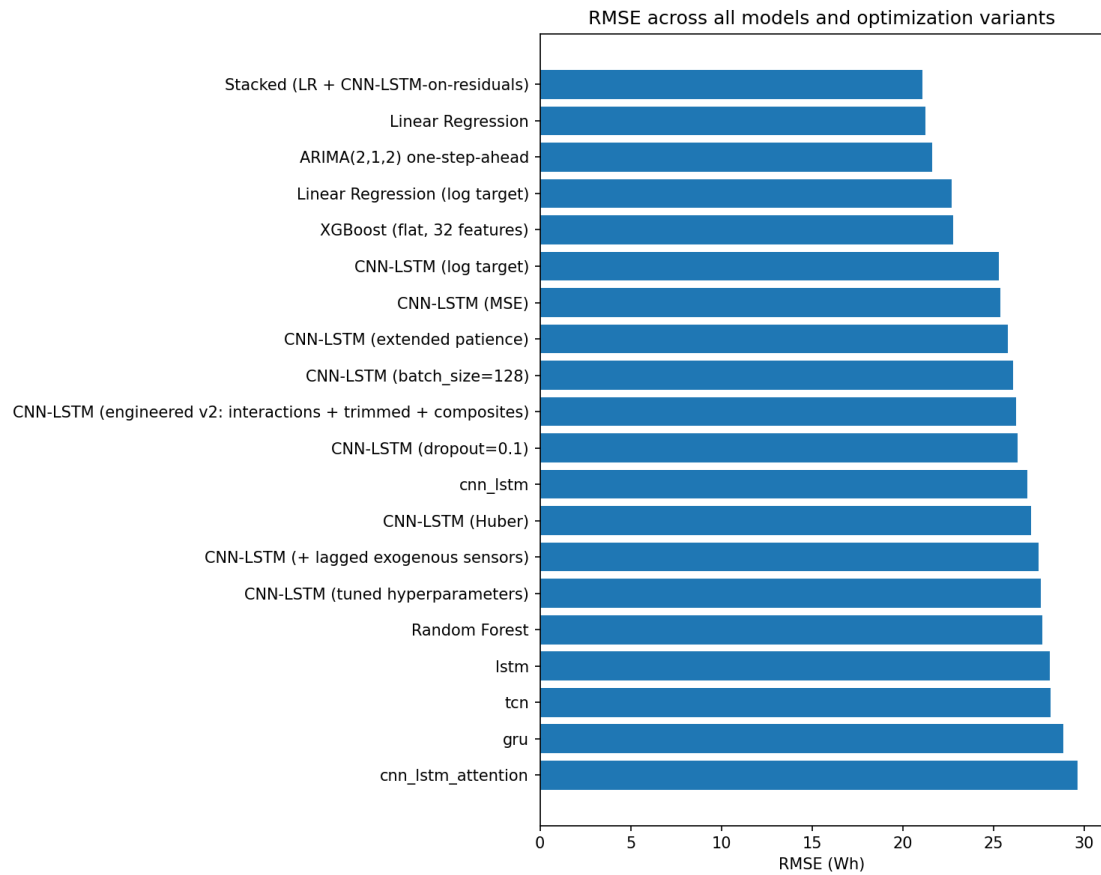


Figure 15: RMSE across every model and optimization variant

7 Model Optimization

7.1 Standard Optimization Techniques

The sequence window length was tuned first where each architecture was swept across candidate windows (24, 72, 144 steps) with a reduced training budget, then retrained at its best window with full epochs and patience. This “sweep then finalize” approach kept the total compute manageable. Three architectures (CNN-LSTM, LSTM, GRU) landed on 72 steps, while TCN and CNN-LSTM with Attention landed on 24 steps.

A proper hyperparameter search was run on CNN-LSTM, the strongest of the five deep models. This was a random search over hidden size (32, 64, 128), CNN channel count (16, 32, 64), and learning rate (5e-4, 1e-3, 2e-3) - CNN-LSTM’s capacity is set by hidden size and channel count rather than a layer count the way GRU’s is - with 6 sampled configurations at a reduced budget, followed by a full-budget retrain of the best one (hidden size 128, 32 channels, learning rate 2e-3). That configuration reached RMSE 27.61, slightly worse than the untuned default (26.87): the validation-loss winner from the search didn’t turn out to generalize better on the test set here.

For regularization spatial dropout was used at a rate of 0.2 across every recurrent and convolutional model by default, for the reasons covered in Model Design. For early stopping, patience-based stopping was used with the best weights restored, and it was also tested as an extended-patience variant (patience of 20 over 100 epochs, compared to the default patience of 10 over 60 epochs) explicitly, to check whether the models were simply cut off too early - this gave CNN-LSTM a meaningful improvement (RMSE 27.07 to 25.79, about 4.7% better).

Batch size and dropout rate were tested separately too, since the brief lists batch size alongside learning rate/layers/neurons under hyperparameter tuning, and dropout rate under regularization, and neither had been touched by the search above. A sweep across batch sizes {32, 64, 128, 256} on the default CNN-LSTM found 128 as the best, a modest improvement (RMSE 26.87 to 26.09, about 2.9% better). A separate sweep across dropout rates {0.1, 0.2, 0.3, 0.4, 0.5}, also on the default CNN-LSTM, found that 0.1 does slightly better than the 0.2 rate locked in during Model Design (RMSE 26.87 to 26.35, about 1.9% better) - a plausible explanation is that the convolution layer already imposes some regularizing structure before the LSTM sees the sequence, so this architecture needs less additional dropout than a plain recurrent model would.

7.2 Investigating Why Deep Learning Underperforms

Rather than treating Linear Regression’s win as something to gloss over, a series of independent tests were run to figure out whether that gap could still be closed, or whether it reflects something real about the data.

The first test was a raw-features-only ablation. All 11 of the **Appliances**-derived lag and rolling features were dropped, keeping only the other 25, and retrained CNN-LSTM. Its R2 dropped from 0.51 to essentially zero (-0.001), barely better than predicting the mean every time. That’s a strong signal that almost all of the usable signal in this dataset is autoregressive.

Next ARIMA(2,1,2) was tested, using only the target’s own history and no features at all. It reached an RMSE of 21.64 and R2 of 0.686, close behind Linear Regression and the Stacked model. That showed the ceiling isn’t specific to deep learning or to multivariate modeling generally, since a classical univariate model with none of the engineering effort gets close to the same place.

XGBoost was also tested on the same 36 features, since it’s non-linear but not sequential. It

reached RMSE 22.78 and R2 0.652, clearly ahead of every standalone deep learning model but behind Linear Regression and ARIMA. Non-linear tree interactions still don't close the gap once `Appliances_lag1` is available as an input.

Alongside this a VIF (variance inflation factor) analysis was run, since the pairwise correlation filter in feature selection can miss group-level multicollinearity. It found several features with severe collinearity: `Tdewpoint` at 329, `indoor_outdoor_delta` at 151, `RH_out` at 115, and `heat_index_proxy` at 75, since several of the raw temperature sensors are highly correlated with each other and with `Tdewpoint`. This matters for how reliable Linear Regression's individual coefficients are to interpret, but it doesn't explain the accuracy gap, since these are exactly the weak-signal environmental features the ablation test already showed contribute very little.

Beyond the standard optimization levers already covered above (window, hyperparameter search, extended patience, batch size, dropout), three more targeted changes were tried on CNN-LSTM. An updated feature set (interaction terms between the lag-1 feature and time of day, a trimmed lag set, and composite indoor temperature/humidity features) reached RMSE 26.25, a real if modest improvement (about 3% better than the same-window Huber baseline of 27.07). A log-transform of the target had a mixed effect, as expected from the skew: it made Linear Regression noticeably worse (RMSE 21.25 to 22.69) but helped CNN-LSTM substantially (27.07 down to 25.30, about 6% better) - the single largest standalone improvement tried on this model. Adding lagged versions of the outdoor temperature and humidity sensors, to test for a delayed thermal-inertia effect, left CNN-LSTM slightly worse (27.07 to 27.49), no evidence of a useful signal there.

The last and most important experiment was stacking. The approach: is to fit Linear Regression, compute its residuals on the training set, and train a CNN-LSTM specifically to predict those residuals rather than the raw target, then combine both predictions at test time. This reaches RMSE 21.08 and R2 0.700, the best result in this notebook, edging out Linear Regression alone (21.25, 0.698). The same approach with GRU instead of CNN-LSTM does not beat Linear Regression, which points to why this combination in particular works: the small amount of nonlinear structure left in Linear Regression's residuals seems to need an architecture with local pattern extraction ahead of its sequence model (CNN-LSTM's convolution layer feeding its LSTM) to be captured reliably, rather than being accessible to any recurrent architecture.

Here's the summary of all nine tests:

#	Experiment	Result	What it tells
1	Raw-features-only ablation	CNN-LSTM R2 drops from 0.51 to -0.001	Nearly all the signal is autoregressive
2	ARIMA(2,1,2), target history only	RMSE 21.64, R2 0.686, close behind the two leaders	The ceiling isn't specific to deep learning
3	XGBoost, same 36 features	RMSE 22.78, R2 0.652, ahead of every DL model but behind the leaders	Non-linear interactions don't close the gap
4	VIF analysis	Several features show severe multicollinearity	Affects interpretability, not accuracy

#	Experiment	Result	What it tells
5	CNN-LSTM hyperparameter search	RMSE 26.87 to 27.61 (slightly worse)	Validation-loss winner didn't generalize better
6	Engineered feature set v2	RMSE 27.07 to 26.25, about 3% better	A real but modest gain
7	Log-transformed target	Hurts Linear Regression, helps CNN-LSTM by about 6%	The single biggest standalone lever tried
8	Lagged exogenous sensors	RMSE 27.07 to 27.49 (slightly worse)	No useful thermal-inertia signal found
9	Stacking (Linear Regression plus CNN-LSTM on residuals)	RMSE 21.25 to 21.08, beats Linear Regression	The one approach that closes the gap

Three different methods (the ablation test, ARIMA, and the shared ceiling across the standalone deep learning architectures) point to the same conclusion from different angles: **Appliances** consumption at 10-minute resolution behaves close to a pure autoregressive process, and no amount of extra feature engineering, added model capacity, or target transformation closes the gap for a standalone deep sequence model. What does close it, narrowly, is combining Linear Regression with a CNN-LSTM trained specifically on what Linear Regression gets wrong, rather than replacing Linear Regression outright. Linear Regression is still the best single model tested; the small additional gain from stacking is real but modest, about a 0.8% reduction in RMSE.

8 Challenges and Solutions

Avoiding data leakage turned out to need several separate safeguards, not just one. Rolling and lag features are computed on the target after shifting it by one step, so a feature at row t never sees the value being predicted at that same row. Both scalers are fit only on the training split. And the train/test split itself is chronological rather than random. Each of these were explicitly checked rather than assuming they were handled correctly by default.

The target’s skew of 3.39 shaped a lot of downstream decisions. Huber loss was chosen over MSE specifically because of it, and later tested that choice against an explicit log-transform of the target. The log-transform actually hurt Linear Regression, since it distorts the otherwise close to linear relationship between `Appliances_lag1` and the raw target, even though it clearly helped CNN-LSTM (see Model Optimization). That confirmed Huber loss was the better fix specifically for the linear model, even though a log target turned out to be worth using for the deep model.

Feature selection needed more than one method to trust. Random Forest importance ranked `Press_mm_hg` suspiciously high, and permutation importance, which measures actual held-out performance impact rather than tree-impurity reduction, showed that ranking was an artifact and dropped it. If it was stopped after the Random Forest stage, a non-predictive feature would have made it into the final set.

A more serious problem was found in that same Stage 4 permutation importance step, after a systematic audit of every function that touches train/test data. The audit checked every function in `src/data_preprocessing.py`, `src/feature_engineering.py`, and `src/train.py`, and found three places where the full dataset, rather than the training split alone, technically influenced a preprocessing decision. Two of them, the IQR outlier fence and the Stage 2 correlation-based pruning, were verified empirically to have zero practical effect: recomputing both using only the training split produced identical fence values and identical dropped features. The third, Stage 4 permutation importance, was different and real. It measured feature importance directly on the test set rather than a held-out validation split, letting the actual test set influence which features survived into the final model, then reusing that same test set to report every downstream metric, a genuine feature-selection leak. Fixing it (carving an 85/15 validation split from training data instead) changed the selected feature count from 32 to 36 and materially changed which second-tier features were kept. Every notebook from Feature Engineering onward was rerun after the fix. The practical impact turned out to be uneven. Random Forest and the Stacked model, the two results that had benefited most from the leak, both got noticeably worse once it was corrected, while Linear Regression and ARIMA barely moved, which is itself further evidence for how little the environmental features and their interactions matter next to the target’s own autoregressive structure.

Fixing Stage 4 also changed which deep learning architecture was strongest. CNN-LSTM overtook GRU on the corrected feature set. The optimization notebook’s experiments (hyperparameter search, batch size and dropout sweeps, the log-transform and exogenous-lag tests, and the residual-stacking hybrid) all target whichever architecture is strongest, so all of them were retargeted from GRU to CNN-LSTM to match. That retargeting turned out to matter for one result specifically: stacking Linear Regression with a GRU on residuals does not beat Linear Regression, but stacking with CNN-LSTM does, by a small margin. Using the wrong “best” architecture for that experiment would have missed the only result in the whole notebook that actually improves on Linear Regression.

Deep learning not beating the baseline could have easily been written off as a failure, but methods were tried to treat it as something worth investigating properly instead. The nine tests in Model Optimization exist specifically to tell apart “the models are undertrained, undersized, or missing features” from “the ceiling is real,” and the combination of the ablation test with the ARIMA and XGBoost comparisons gives fairly strong, converging evidence for the second explanation.

A less expected problem came up when adding the XGBoost comparison model. It crashed the notebook kernel outright. PyTorch and XGBoost each bundle their own copy of the OpenMP threading runtime, and loading both into the same process crashes once PyTorch has already done real training work. Setting `n_jobs=1` on XGBoost’s regressor reduced the problem but didn’t fully fix it. The actual fix was to run XGBoost in a completely separate subprocess (`src/run_xgboost_subprocess.py`) that never imports `torch` at all, reading its result back as JSON. Keeping the two libraries in separate processes avoided the conflict entirely instead of trying to make them coexist in one.

That crash also highlighted something worth remembering about notebooks in general. When a kernel dies, everything in memory goes with it, not just whatever the current cell was doing. A `.ipynb` file only saves the code and the last output of each cell, not the live variables sitting in the kernel’s memory while it runs. So there’s no way to “resume” a crashed notebook partway through. The only reliable fix was a full, clean rerun of the whole notebook from the top.

9 Conclusion

The best model overall is a hybrid. Linear Regression, followed by a CNN-LSTM trained on Linear Regression’s own residuals, with both predictions combined at test time. It reaches MAE 13.52, RMSE 21.08, and R2 0.700, a small but real improvement over plain Linear Regression alone (13.60, 21.25, 0.698), which is itself the strongest single model tested, ahead of all five deep learning architectures, a classical ARIMA model, and a gradient-boosted tree ensemble. The margin the hybrid wins by is narrow, about a 0.8% reduction in RMSE, but it is a genuine result: stacking with GRU instead of CNN-LSTM does not produce the same win, which suggests the small amount of nonlinear structure left in Linear Regression’s residuals specifically needs an architecture that extracts local patterns before modeling the sequence, not just any recurrent model.

The key finding is that **Appliances** consumption at 10-minute resolution behaves close to a pure autoregressive process. This must not be claimed from a single result, so it was checked several separate ways. A raw-features ablation that drops to an R2 of roughly zero without the target’s own lag and rolling history, a univariate ARIMA model with no feature engineering at all that lands close behind the two leaders, a non-sequential gradient-boosted model (XGBoost) on the same features that still trails both, and a shared performance ceiling across all five standalone deep learning architectures that neither more capacity, better features, nor a target transformation meaningfully closes. The one thing that does close the remaining gap is combining Linear Regression with a model built specifically to learn what it gets wrong, not a bigger or better-tuned version of Linear Regression’s replacement.

But it cannot be concluded that the deep learning side of the project failed. The training curves show healthy convergence with no overfitting, five distinct architectures were tested, and a genuine hyperparameter search was run rather than just tweaking numbers by hand. It’s more a property of this specific dataset at this specific resolution. With a dominant AR(1) signal and fairly weak correlation from the environmental sensors, a closed-form linear fit on **Appliances_lag1** is hard to beat by learning the same relationship through gradient descent on around 13,300 training rows. The one approach built specifically to exploit any leftover nonlinear structure, stacking a CNN-LSTM on Linear Regression’s residuals, does manage a small improvement, which is itself informative: there is a real, if modest, amount of signal left over that a purely linear model cannot reach on its own.

A few things that should be tried if kept working on this:

More data would help. 137 days gives around 13,300 effective training rows once you account for the feature-engineering drop and windowing, which is a modest sample for a deep sequence model. A longer collection period might let the extra capacity of the deep models actually pay off.

An attempt was made on seasonal ARIMA/SARIMAX model at the daily period (144 steps) but found it too slow to run at this dataset’s scale within the time and computation power had. It would be worth revisiting with a faster implementation.

The stacking approach beats Linear Regression by a narrow margin, and it’s worth being honest that a narrow win on one held-out test set doesn’t fully settle the question of how much genuine nonlinear residual structure exists. A more careful residual model, with cross-validated stacking rather than a single train/test split, would give a more confident answer either way.

It would be also better to properly test the holiday and special-event features the brief suggests, using a verified calendar for the house’s location and dates, rather than leaving them out for lack

of a reliable source.

And if per-appliance or sub-metered data were available, it might reveal structure that the aggregate **Appliances** reading hides completely.

10 References

1. Candanedo, L. M., Feldheim, V., and Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81-97. This is the original paper behind the UCI Appliance Energy Prediction dataset used in this assessment.
2. TensorFlow: Time Series Forecasting with LSTM Neural Networks (the tutorial referenced in the assessment brief).
3. Kaggle: Time Series Feature Engineering (the guide referenced in the assessment brief).
4. Chollet, F. Deep Learning with Python (referenced in the assessment brief as a general architecture and training reference).
5. PyTorch’s official documentation and tutorials, for the `torch.nn` layer reference, the `DataLoader/Dataset` API, `nn.HuberLoss`, and `CosineAnnealingLR`.
6. Scikit-learn’s documentation for `LinearRegression`, `RandomForestRegressor`, `StandardScaler`, `MinMaxScaler`, and `permutation_importance`.
7. Statsmodels documentation for `STL`, `ARIMA`, and `variance_inflation_factor`.
8. Bai, S., Kolter, J. Z., and Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. This is the paper behind the Temporal Convolutional Network architecture implemented in `src/model.py`.

AI tools declaration: I used Claude during this assessment for architecture planning, code structure and docstring guidance, debugging (including tracking down the PyTorch/XGBoost crash described in Challenges and Solutions), and planning the optimization experiments in notebooks/05_Optimization_Evaluation.ipynb. I reviewed and understood every implementation, analysis, and final decision myself with proper justifications.